DD: FUL-02-STEP-INSTALL — Install Work Order

Developer implementation guide: `DD_FUL-02-STEP-INSTALL-DEV_IMPL_GUIDE-v1.0.md`.

1. Purpose

This rewritten DD is the developer-facing version of the module contract `DD_FUL-02-STEP-INSTALL-v1.0.md`.

Verdict: ■ New | **Wave:** 1 | **Group III:** Fulfillment | **Version:** v1.0 | **Satellite of:** DD_FUL-02-Order_Capture-v1.0.md
Dispatches a contractor to physically install the customer's equipment (ONT, ATA, decoder, etc.), wires the cabling, captures equipment serial numbers, and reports the WO finalized. Activation follows. Phases: `READY_FOR_INSTALL` → `INSTALLING` → optionally `INSTALL_FAILED`.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	`/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/06_fulfillment/DD_FUL-02-STEP-INSTALL-v1.0.md`
Version	`v1.0`
DD type	`module contract`
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- `GET /api/packages/pkg_INET_VOIP_RES`
- `GET /api/packages/{ref}`
- `GET /api/services/by-code/svc_INET_GPON_100M`
- `GET /api/services/by-code/{code}`
- `POST /api/work-orders`
- `POST /api/work-orders/wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2/attachments`
- `POST /api/work-orders/wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2/cancel`
- `POST /api/work-orders/wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2/equipment-bindings`

- `POST /api/work-orders/wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2/finalize-first-confirm`
- `POST /api/work-orders/wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2/finalize-second-confirm`
- `POST /api/work-orders/wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2/notes`
- `POST /api/work-orders/{woId}/cancel`
- `POST /api/work-orders/{woId}/equipment-bindings`
- `POST /engine-rest/message`

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- `create-install-wo`
- `mark-install-failed`
- `order-create-install-wo`
- `order-mark-install-failed`
- `order-trigger-activation`
- `trigger-activation`
- `wait-install-finalize`
- `wo-auto-assign-contractor`
- `wo-capture-equipment-bindings-at-finalize`

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- `WorkOrderAssigned`
- `WorkOrderCancelled`
- `WorkOrderCancelledConsumer`
- `WorkOrderCreated`
- `WorkOrderFinalized`
- `WorkOrderFinalizedConsumer`
- `WorkOrderInstallationCompleted`

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.

- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 1 | **Group III:** Fulfillment | **Version:** v1.0 | **Satellite of:** DD_FUL-02-Order_Capture-v1.0.md

Dispatches a contractor to physically install the customer's equipment (ONT, ATA, decoder, etc.), wires the cabling, captures equipment serial numbers, and reports the WO finalized. Activation follows. Phases: `READY_FOR_INSTALL` → `INSTALLING` → optionally `INSTALL_FAILED`.

1. What this step does

The order has been validated, the subscription exists, the customer has paid, and (in WIK) KYC has cleared. Now the actual physical work: send a technician to the customer's address with the right equipment, install it, configure the customer's onboard hardware, capture serial numbers so we can later activate them on the network.

The step is structured around two outbound things (the worker creating the WO) and three inbound things (Kafka messages that can arrive while we wait):

1. **Create the install WO** (`order-create-install-wo` worker). The worker figures out what equipment the customer needs (by walking the Package → Services → each addressable Service's `equipmentRequirementRef`), then asks the WO module to create a Work Order via `POST /api/work-orders` with `kind=INSTALLATION` and the appropriate `jobTypeCode` (H01 for HFC, GIN for GPON internet, GTV for GPON TV, G07 for VOIP, derived from the package's network type and service mix). The WO module's Installation flow (DD_WO-01-FLOW-INSTALLATION) handles

contractor assignment via Drools, dispatches the visit, and emits `WorkOrderCreated`. The worker captures `woId` onto the order row.

2. **Wait for the WO to finalize** (`WorkOrderFinalized` message catch). The BPMN parks at `wait-install-finalize` with a PT30D boundary timer. The contractor visits, does the work, captures equipment bindings via the WO module's `POST /api/work-orders/{woId}/equipment-bindings` endpoint, runs through the WO Installation flow's 2-step finalize (first-confirm + second-confirm with checklist enforcement + Drools client-up gate), and the WO closes COMPLETED. The WO module emits `WorkOrderFinalized` on `sophix.work-orders.finalized`.
3. **Or wait for the WO to be cancelled** (`WorkOrderCancelled` message catch). Either parallel to or as a boundary on the finalize wait, depending on operator BPMN. A WO can be cancelled by the operator (operational decision), by the contractor (unable to fulfil — building won't grant access), or as a cascade from STEP-CANCELLATION (customer cancelled the order). Different from finalization — the WO is closed without delivering. WO module emits `WorkOrderCancelled` on `sophix.work-orders.cancelled`.
4. **Or the PT30D timer fires** — no finalize within 30 days. Goes to `order-mark-install-failed` with `INSTALL_TIMEOUT`. Worker calls `POST /api/work-orders/{woId}/cancel` to clean up the WO module side.

When `WorkOrderFinalized` arrives, our consumer parses the equipment bindings carried in the event (the actual ONT serial number that was installed at this customer's address, etc.), writes them onto `customer_order.equipment_bindings_captured`, and wakes the BPMN. The BPMN advances to STEP-ACTIVATION, which uses these bindings to know which physical hardware to activate on the network.

Note: WO module is real in v1.0. The earlier `install_wo_stub` placeholder is retired. STEP-INSTALL calls the WO Installation flow directly; the WO module owns contractor assignment, equipment binding capture, client-up verification, topology confirmation, and warranty tagging. See DD_WO-01-Work_Order_Module-v1.0 for the constellation.

Equipment resolution

The Package references a set of Services (via SIP-01). Each Service has a flag `isAddressable`. Addressable Services require physical equipment to deliver; each carries an `equipmentRequirementRef` like `GPON_ONT` or `ATA`. Non-addressable Services (e.g., a billing-only "premium support" Service) don't.

For a triple-play residential package `pkg_INET_VOIP_RES` with services `svc_INET_GPON_100M` (`isAddressable=true`, `equipmentRequirementRef=GPON_ONT`) and `svc_VOIP_HOSTED` (`isAddressable=true`, `equipmentRequirementRef=ATA`), the worker resolves equipment requirements `[GPON_ONT, ATA]`. The install WO carries this list; the contractor knows to bring both pieces of hardware.

What happens when the contractor reports back

The contractor app posts a finalize call with the equipment bindings — for each requirement, the equipmentRef (a record in the equipment inventory — out of FUL-02 scope) and the serial number physically labelled on the unit installed at the customer's home. The WO module captures this and emits `WorkOrderFinalized` carrying the bindings. Our consumer writes them onto the order, then wakes Camunda — STEP-ACTIVATION later passes them to SUB-WF-ACTIVATE-01, which passes them to FUL-03, which uses them to know "activate ONT serial HW123456 on OLT NRB-KAREN port 3-7."

WorkOrderCancelled handling — operator policy choice

When the WO is cancelled without finalizing, what should happen to the order? Two reasonable answers:

- **Auto-fail the order** (`INSTALL_FAILED`, refund if paid). Simple, cleanly closes the loop. WIK BPMN v1.0 does this.
- **Open a manual investigation user task** — let ops decide whether to create a new WO (keep the order alive) or fail it. More flexible but requires staff judgment per case.

Operator BPMNs choose. v1.0 ships WIK with auto-fail; documented for future operators that might want manual review.

What this step deliberately doesn't do

- **Doesn't dispatch the contractor itself**. The WO module owns contractor routing (tech region → contractor slot → team slot per workshop). We just create the WO.
- **Doesn't activate equipment on the network**. STEP-ACTIVATION → SUB-WF-ACTIVATE-01 → FUL-03 owns network activation. We capture the equipment serial numbers and pass them through.
- **Doesn't physically verify the install**. Trust the contractor's finalize call. If the contractor fakes it, downstream signals (activation failure on FUL-03 because the serial number doesn't map to the network port) will catch it later.

- **Doesn't auto-retry a failed install**. INSTALL_FAILED is terminal. Customer gets a refund and submits a new order if they want to try again.

2. Worked example end-to-end (WIK, FEASIBLE → FINALIZED)

Continuing John Mwangi's order from STEP-KYC. Order `ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1`, customer `cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6`, package `pkg_INET_VOIP_RES`, subscription `sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q`, homepass `hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P`. KYC cleared at 11:42:33; Camunda just advanced to `create-install-wo`.

Step 1 — Worker resolves equipment requirements

`CreateInstallWoWorker` picks up the external task. Process variables: `orderId`, `customerId`, `homepassId`, `packageRef = pkg_INET_VOIP_RES`, `subscriptionId = sub_01HX...Q`, `operatorCode = WIK`.

Worker first fetches the Package from SIP-01 to get the Service refs:

```
GET /api/packages/pkg_INET_VOIP_RES HTTP/1.1
Host: sip-01.sophix.internal
```

Response (abridged to the relevant field):

```
{
  "packageRef":  "pkg_INET_VOIP_RES",
  "serviceRefs": ["svc_INET_GPON_100M", "svc_VOIP_HOSTED"]
}
```

For each service ref, worker fetches the Service from PLM-CFG-01:

```
GET /api/services/by-code/svc_INET_GPON_100M HTTP/1.1
Host: plm-cfg-01.sophix.internal
```

Response:

```
{
  "code":                   "svc_INET_GPON_100M",
  "isAddressable":          true,
  "equipmentRequirementRef":"GPON_ONT",
  "networkProfileShape":    { "provisionerKey": "gpon-provisioner-v1", "params": { "speedMbps": 100 } },
  "serviceManagementKey":   "provisioning",
  "consumptionModel":       "CONTINUOUS"
}
```

Same for `svc_VOIP_HOSTED`:

```
{
  "code":                   "svc_VOIP_HOSTED",
  "isAddressable":          true,
  "equipmentRequirementRef":"ATA",
  "networkProfileShape":    { "provisionerKey": "voip-provisioner-v1", "params": { "lineCount": 1 } },
  "serviceManagementKey":   "voice",
  "consumptionModel":       "CONTINUOUS"
}
```

Worker aggregates into a requirements list: `[{type: GPON_ONT, mandatory: true}, {type: ATA, mandatory: true}]`.

Step 2 — Worker creates the install WO

The worker first derives the WO's `jobTypeCode` from the package's network type and service mix:

- HFC + internet → `H01`
- GPON + internet → `GIN`
- GPON + TV → `GTV` (separate WO if both internet + TV; v1.0 ships one WO per service line)
- VOIP only → `G07`

For the worked example, the package is `pkg_INET_VOIP_RES` (GPON internet + VOIP). Two WOs would be needed (one GIN, one G07) — but for simplicity here, we walk through the GIN install case. The VOIP install runs in parallel as a sibling WO with its own correlation.

```
POST /api/work-orders HTTP/1.1
Host: work-order.sophix.internal
Authorization: Bearer <ful-02-svc-creds>
Content-Type: application/json
```

```
{
  "operatorCode":           "WIK",
  "kind":                   "INSTALLATION",
  "jobTypeCode":            "GIN",
  "customerId":             "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
  "homepassId":             "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P",
  "originatingContextType": "ORDER",
  "originatingContextRef":  "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "idempotencyKey":         "ful-02-step-install-ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1-GIN",
  "metadata": {
    "subscriptionId":         "sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q",
    "packageRef":             "pkg_INET_VOIP_RES",
    "customerName":           "Jane Mwangi",
    "contactNumber":          "+254712345678",
    "techRegionCode":         "NRB-KAREN",
    "equipmentRequirements":  [
      { "type": "GPON_ONT", "mandatory": true }
    ]
  }
}
```

Note: the equipment requirements list is carried in the WO's `metadata` JSONB for the dispatcher to see what to bring. Actual equipment bindings (the serials physically installed) are captured at finalize time via the WO module's `POST /api/work-orders/{woId}/equipment-bindings` endpoint, NOT at WO creation. The contractor's BO UI shows the requirements from `metadata` so the dispatcher knows to load the right hardware on the truck.

The WO module's Installation flow validates the request (R-WO-FW-1 idempotency on `(operator, originating_context_type, idempotency_key)`), starts a Camunda process instance for `WorkOrder_Installation_WIK.bpmn`, runs the `wo-auto-assign-contractor` external task (Drools picks `ctr_Z071H + team_rapid_response` for NRB-KAREN GPON installs per the rules in DD_WO-01-FLOW-INSTALLATION §4.1), updates status to ASSIGNED, and returns:

```
HTTP/1.1 201 Created
Content-Type: application/json

{
  "woId":               "wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2",
  "operatorCode":       "WIK",
  "kind":               "INSTALLATION",
  "jobTypeCode":        "GIN",
  "status":             "ASSIGNED",
  "currentPhase":       null,
  "assignmentMode":     "TEAM",
  "assignedContractorId":"ctr_Z071H",
  "assignedTeamId":     "team_rapid_response",
  "assignedTechnicianId":null,
  "scheduledAt":        null,
  "createdAt":          "2026-05-18T12:00:00.123Z"
}
```

The WO module emits `WorkOrderCreated` on `sophix.work-orders.created` and `WorkOrderAssigned` on `sophix.work-orders.assigned`. The Installation flow's BPMN then proceeds: a team member claims via `/claim`, schedules the visit, the contractor's BO UI surfaces the WO for dispatch.

Worker UPDATEs:

```
UPDATE customer_order
   SET work_order_id    = 'wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2',
       last_updated_at  = NOW()
 WHERE order_id         = 'ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1';
```

Worker emits `sophix.orders.install-dispatched`:

```
{
  "eventType":   "OrderInstallDispatched",
  "aggregateId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "timestamp":   "2026-05-18T12:00:00.234Z",
  "payload": {
    "orderId":               "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "customerId":            "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
    "operatorCode":          "WIK",
    "subscriptionId":        "sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q",
    "workOrderId":           "wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2",
    "homepassId":            "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P",
    "packageRef":            "pkg_INET_VOIP_RES",
    "equipmentRequirements": [
      { "type": "GPON_ONT", "mandatory": true },
      { "type": "ATA",      "mandatory": true }
    ],
    "scheduledFor":          "2026-05-18T15:00:00Z",
```

```
    "installTimeoutAt":      "2026-06-17T12:00:00Z",
    "dispatchedAt":          "2026-05-18T12:00:00.234Z"
  }
}
```

DD_NOT-01 consumes it and SMSes John: "Your Zuku Fiber install is scheduled for Sat 18 May, 3pm. Our technician will arrive within a 1-hour window."

Worker completes the task. Camunda advances to `wait-install-finalize` (message catch event with PT30D boundary timer).

`PhaseProjector` updates:

```
UPDATE customer_order
   SET current_phase       = 'INSTALLING',
       current_activity_id = 'wait-install-finalize',
       last_updated_at     = NOW()
 WHERE order_id            = 'ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1';
```

(In WIK BPMN, `READY_FOR_INSTALL` and `INSTALLING` collapse: the BPMN goes directly to `wait-install-finalize` after `create-install-wo`. The `READY_FOR_INSTALL` phase in the UNION enum exists for operator BPMNs that may interpose additional steps between WO creation and wait — none in v1.0.)

Step 3 — Contractor finalizes (3 hours later)

The contractor's BO UI follows the WO Installation flow's full finalize discipline (per DD_WO-01-FLOW-INSTALLATION §12.1 walkthrough). The tech `tech_01HZ_KAREN_03` claimed the WO, drove to site, and installs the ONT (serial `HW_ONT_NK4501`). Tests local connectivity (ONT shows green PON LED, optical RX power within threshold). Captures the findings, solution, optical readings, client-up evidence, topology confirmation, equipment binding, and required attachments via the WO module's standard endpoints:

```
POST /api/work-orders/wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2/notes
  noteKind=findings                      (free-text install summary)
POST /api/work-orders/wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2/notes
  noteKind=solution                      (free-text resolution)
POST /api/work-orders/wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2/notes
  noteKind=optical_readings              (ontRxDbm, oltRxDbm, distanceM, etc.)
POST /api/work-orders/wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2/notes
  noteKind=client_up_evidence            (evidenceType, comments)
POST /api/work-orders/wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2/notes
  noteKind=network_topology_confirmation  (confirmedAsPlanned)
POST /api/work-orders/wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2/attachments  (setup_photo)
POST /api/work-orders/wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2/attachments  (rf_optical_reading)
POST /api/work-orders/wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2/equipment-bindings
{
  "bindings": [{
    "equipmentKind": "GPON_ONT",
    "brand":         "Huawei",
    "model":         "HG8245H",
    "serialNumber":  "HW_ONT_NK4501",
    "macAddress":    "00:1A:2B:3C:4D:5E",
    "onuId":         "12",
    "action":        "INSTALLED",
    "notes":         "Mounted on living room wall, near power outlet"
  }],
  "recordedBy":     "tech_01HZ_KAREN_03",
  "recordedByRole": "FIELD_TECH"
}
```

Then runs the WO Installation flow's 2-step finalize:

```
POST /api/work-orders/wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2/finalize-first-confirm
{ "finalReasonCode": "INSTALL_COMPLETED_CUSTOMER_UP", "confirmedBy": "ctr_Z071H_dispatcher_01" }

(Drools client-up-decision passes; equipment-bindings worker passes)

POST /api/work-orders/wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2/finalize-second-confirm
```

The WO module's checklist enforcement passes (all required notes + attachments present per `wo_finalization_requirements` for `(WIK, INSTALLATION, GIN)`). WO transitions to COMPLETED. WO module emits:

```
{
  "eventType":   "WorkOrderFinalized",
  "aggregateId": "wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2",
  "timestamp":   "2026-05-18T15:35:00.412Z",
  "payload": {
    "woId":                    "wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2",
    "operatorCode":            "WIK",
```

```
    "kind":                    "INSTALLATION",
    "jobTypeCode":             "GIN",
    "networkType":             "GPON",
    "customerId":              "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
    "homepassId":              "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P",
    "originatingContextType":  "ORDER",
    "originatingContextRef":   "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "completedAt":             "2026-05-18T15:35:00.000Z",
    "finalReasonCode":         "INSTALL_COMPLETED_CUSTOMER_UP",
    "finalAssignedContractorId": "ctr_Z071H",
    "finalAssignedTeamId":     "team_rapid_response",
    "finalAssignedTechnicianId": "tech_01HZ_KAREN_03",
    "equipmentBindings": [
      {
        "equipmentRefId":  "eqref_01HZ_A1",
        "equipmentKind":   "GPON_ONT",
        "brand":           "Huawei",
        "model":           "HG8245H",
        "serialNumber":    "HW_ONT_NK4501",
        "macAddress":      "00:1A:2B:3C:4D:5E",
        "onuId":           "12",
        "action":          "INSTALLED"
      }
    ]
  }
}
```

Published on topic `sophix.work-orders.finalized`. The WO module also emits the richer flow-specific `WorkOrderInstallationCompleted` event on `sophix.work-orders.installation-completed` carrying `warrantyExpiresAt` + `topologyResolution`. STEP-INSTALL's consumer subscribes to `WorkOrderFinalized` (sufficient for activation); future enrichment (e.g., letting STEP-ACTIVATION read the resolved topology) can switch to consuming `WorkOrderInstallationCompleted` instead.

Step 4 — Our consumer wakes Camunda

`WorkOrderFinalizedConsumer` receives the event:

1. Look up the order by `work_order_id = 'wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2'` → matches `ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1`, phase `INSTALLING`. (The consumer also filters by `payload.originatingContextType == 'ORDER'` to avoid picking up events from non-FUL-02-created WOs.)
2. Idempotency check on `(operatorCode, woId)`: not seen yet.
3. Update the order with the captured bindings (carrying the WO module's richer schema forward):

```
UPDATE customer_order
   SET equipment_bindings_captured = '[
         {"equipmentKind":"GPON_ONT","brand":"Huawei","model":"HG8245H",
          "serialNumber":"HW_ONT_NK4501","macAddress":"00:1A:2B:3C:4D:5E",
          "onuId":"12","action":"INSTALLED"}
       ]'::jsonb,
       last_updated_at = NOW()
 WHERE order_id        = 'ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1';
```

4. Record idempotency: INSERT into `customer_order_processed_workorders`.
5. Wake Camunda:

```
POST /engine-rest/message HTTP/1.1
{
  "messageName":  "WorkOrderFinalized",
  "businessKey":  "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "processVariables": {
    "equipmentBindings": {
      "value": "[{\"equipmentKind\":\"GPON_ONT\",\"brand\":\"Huawei\",\"model\":\"HG8245H\",\"serialNumber\":\"HW_ONT_N
      "type": "Json"
    }
  }
}
```

Camunda wakes the parked instance, advances past `wait-install-finalize` to STEP-ACTIVATION's `order-trigger-activation` external task. The `equipmentBindings` process variable carries forward — STEP-ACTIVATION will pass it to SUB-WF-ACTIVATE-01.

Total elapsed in step: ~3 hours 35 minutes (most of it the contractor's drive + work).

3. Worked example — WorkOrderCancelled (WIK auto-fail)

Variant: at 13:30 the contractor reports he can't reach the address (closed gated community, no security access). Operations marks the WO `CANCELLED` via the WO module's standard cancel endpoint:

```
POST /api/work-orders/wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2/cancel HTTP/1.1

{
  "cancellationReason": "ACCESS_DENIED",
  "cancellationDetail": "Gated community refused contractor access; multiple attempts failed",
  "cancelledBy":        "ops_user_03"
}
```

The WO module's framework cancel handler transitions status to CANCELLED, runs the BPMN's interrupting boundary message event (DD_WO-01-FRAMEWORK §2.4), and emits:

```
{
  "eventType":   "WorkOrderCancelled",
  "aggregateId": "wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2",
  "timestamp":   "2026-05-18T13:30:00.234Z",
  "payload": {
    "woId":                   "wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2",
    "operatorCode":           "WIK",
    "kind":                   "INSTALLATION",
    "jobTypeCode":            "GIN",
    "originatingContextType": "ORDER",
    "originatingContextRef":  "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "cancellationReason":     "ACCESS_DENIED",
    "cancellationDetail":     "Gated community refused contractor access; multiple attempts failed",
    "cancelledBy":            "ops_user_03",
    "cancelledAt":            "2026-05-18T13:30:00Z"
  }
}
```

Published on `sophix.work-orders.cancelled`.

`WorkOrderCancelledConsumer` looks up the order by `work_order_id`, filters by `originatingContextType=ORDER`, sees phase `INSTALLING`, idempotency-checks, calls Camunda:

```
POST /engine-rest/message HTTP/1.1
{
  "messageName":  "WorkOrderCancelled",
  "businessKey":  "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "processVariables": {
    "workOrderCancellationReason": { "value": "ACCESS_DENIED",                                          "type": "String"
    "workOrderCancellationDetail": { "value": "Gated community refused contractor access; multiple attempts failed", "type
  }
}
```

The WIK BPMN has a non-interrupting boundary message event on the `wait-install-finalize` catch that routes `WorkOrderCancelled` to `order-mark-install-failed` (auto-fail per WIK policy). The mark-failed worker writes:

```
UPDATE customer_order
   SET current_phase   = 'INSTALL_FAILED',
       failure_reason  = 'WORK_ORDER_CANCELLED',
       failure_detail  = 'WO wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2 cancelled by ops: ACCESS_DENIED — Gated community refused contr
       last_updated_at = NOW()
 WHERE order_id        = 'ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1';
```

Customer paid → emit `OrderRefundRequested`. Then emit `OrderInstallFailed`. DD_NOT-01 SMSes John: "Sorry, we couldn't complete the install. You'll receive a refund within 5 business days."

4. Worked example — PT30D timeout

If the install hasn't been finalized in 30 days, Camunda's boundary timer fires. Routes to `order-mark-install-failed` with `failureReason = INSTALL_TIMEOUT`, refund cascade if paid.

5. Workers

5.1 order-create-install-wo

Field	Value
Topic	`order-create-install-wo`
Purpose	Resolve equipment requirements from Package + Services, ask the WO module to create a `kind=install` Work Order.

Field	Value
Process variables read	`orderId`, `customerId`, `subscriptionId`, `homepassId`, `packageRef`, `operatorCode`, optional `intendedActivationDate`
External HTTP calls	SIP-01 `GET /api/packages/{ref}` (Package's `serviceRefs[]`). PLM-CFG-01 `GET /api/services/by-code/{code}` per Service (extract `equipmentRequirementRef` where `isAddressable=true`). WO module `POST /api/work-orders` with `kind=INSTALLATION` and derived `jobTypeCode` per DD_WO-01-FRAMEWORK §8.1.
Process variables written	`workOrderId`, `equipmentRequirements` (JSON array)
`customer_order` columns written	`work_order_id`
Kafka emissions	`sophix.orders.install-dispatched`
Lock duration	30 000 ms
Retry policy	3 retries with PT5M between attempts

Failure modes:

- **WO module rejects** (no contractor available, region unsupported) → `/failure` with the error → BPMN routes to `order-mark-install-failed` with `failureReason = INSTALL_WO_CREATION_FAILED`.
- **PLM-CFG-01 / SIP-01 unreachable** → `/failure` with `UPSTREAM_UNREACHABLE`. Incident.
- **Service has no equipment requirement when** `isAddressable=true` → catalog inconsistency; `/failure` with `CATALOG_INCONSISTENT`. Incident.

5.2 order-mark-install-failed

Field	Value
Topic	`order-mark-install-failed`
Purpose	Terminal handler for install path failures (timeout, WO creation failure, WO cancellation). Marks INSTALL_FAILED, emits failure + refund events.
Process variables read	`orderId`, `installFailureReason`, optional `failureDetail`, `operatorCode`
External HTTP calls	None
Process variables written	None (terminal)
`customer_order` columns written	`current_phase = INSTALL_FAILED`, `failure_reason`, `failure_detail`
Kafka emissions	`sophix.orders.install-failed`. If `payment_received_amount > 0`: also `sophix.orders.refund-requested`.
Lock duration	5 000 ms
Retry policy	3 retries with PT30S

6. Message catch events

6.1 WorkOrderFinalized

Triggered by: our `WorkOrderFinalizedConsumer` after receiving `sophix.work-orders.finalized`.

Where BPMNs park: `<bpmn:intermediateCatchEvent>` named `wait-install-finalize`.

Boundary timer: PT30D interrupting. Operator-tunable.

Process variables set on correlation: `equipmentBindings` (Json-typed serialised array).

6.2 WorkOrderCancelled

Triggered by: our `WorkOrderCancelledConsumer` after receiving `sophix.work-orders.cancelled`.

Where BPMNs park: typically a non-interrupting boundary message event on the `wait-install-finalize` catch, OR an interrupting boundary on the `INSTALLING` subprocess scope. Operator-BPMN-defined.

In WIK v1.0: routes to `order-mark-install-failed` with `failureReason = WORK_ORDER_CANCELLED` (auto-fail). Future operators may route to a manual-investigation user task instead.

Process variables set on correlation: `workOrderCancellationReason`, `workOrderCancellationDetail`.

6.3 WorkOrderFinalizedConsumer + WorkOrderCancelledConsumer

Field	WorkOrderFinalizedConsumer	WorkOrderCancelledConsumer
Subscribes to	`sophix.work-orders.finalized`	`sophix.work-orders.cancelled`
Lookup	By `work_order_id` on `customer_order`	Same
Idempotency	`customer_order_processed_workorders(order_id, work_order_id)`	Same
`customer_order` columns written	`equipment_bindings_captured`	None
Camunda call	`POST /engine-rest/message WorkOrderFinalized`	`POST /engine-rest/message WorkOrderCancelled`
DLQ	On no match: log DEBUG, skip. On Camunda 400 (timer fired first): forward to `sophix.orders.message-correlation-dlq`.	Same

7. DB tables touched

customer_order (FRAMEWORK §5.1) — UPDATE

Column	When set	Set by	Example value
`work_order_id`	After WO module accepts creation	`order-create-install-wo`	`wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2`
`equipment_bindings_captured`	When WorkOrderFinalized arrives	`WorkOrderFinalizedConsumer`	JSON array
`current_phase`	On BPMN transitions	`PhaseProjector`	`READY_FOR_INSTALL` → `INSTALLING` → optionally `INSTALL_FAILED`
`current_activity_id`	On each transition	`PhaseProjector`	`create-install-wo` → `wait-install-finalize` → `trigger-activation` (FINALIZED) or `mark-install-failed` (failure)
`failure_reason` (on failure)	`order-mark-install-failed`	`INSTALL_TIMEOUT` / `INSTALL_WO_CREATION_FAILED` / `WORK_ORDER_CANCELLED`	
`failure_detail` (on failure)	`order-mark-install-failed`	Free-text	
`last_updated_at`	On every UPDATE	trigger / consumer / worker	`2026-05-18 15:35:00.612`

Sample data — five orders at various install stages:

order_id	operator_code	work_order_id	equipment_bindings_captured	current_phase	failure_reason
`ord_01HZ...A`	`WIK`	`wo_01HZ...A`	`[{...GPON_ONT...}, {...ATA...}]`	`ACTIVATING` (advancing to STEP-ACTIVATION)	NULL
`ord_01HZ...B`	`WIK`	`wo_01HZ...B`	NULL	`INSTALLING` (WO dispatched, awaiting contractor)	NULL
`ord_01HZ...C`	`WIK`	`wo_01HZ...C`	NULL	`INSTALL_FAILED` (PT30D timeout)	`INSTALL_TIMEOUT`
`ord_01HZ...D`	`WIK`	`wo_01HZ...D`	NULL	`INSTALL_FAILED` (WO cancelled)	`WORK_ORDER_CANCELLED`
`ord_01HZ...E`	`WIK`	`wo_01HZ...E`	`[{...GPON_ONT...}, {...ATA...}]`	`ACTIVATED`	NULL

WO module tables — owned by WO module (FUL-02 does NOT write to these)

After STEP-INSTALL's `order-create-install-wo` worker calls `POST /api/work-orders`, the WO module owns the resulting rows. STEP-INSTALL does not directly INSERT or UPDATE any WO-module table — all interaction is via the WO module's REST API and Kafka events.

Tables owned by the WO module that this step's worker indirectly causes to be populated (per DD_WO-01-FRAMEWORK §6):

- `work_order` — row created with `kind=INSTALLATION, jobTypeCode=GIN|H01|G07|GTV|..., status=PENDING→ASSIGNED→IN_PROGRESS→FINALIZATION_PENDING→COMPLETED|CANCELLED, originating_context_type=ORDER, originating_context_ref=<orderId>`
- `wo_notes` — populated by the contractor during the on-site work (findings, solution, optical_readings, client_up_evidence, network_topology_confirmation, final_reason_set)
- `wo_attachments` — populated by the contractor (setup_photo, rf_optical_reading)
- `wo_equipment_ref_stub` — populated by the contractor at finalize-time with the actual installed serials, MACs, and ONU IDs (see DD_WO-01-FRAMEWORK §9.3; this stub retires when the future Inventory module ships per memory rule #16)
- `wo_status_history`, `wo_assignment_history`, `wo_audit_log` — populated by framework workers

STEP-INSTALL reads from `customer_order` the `work_order_id` for cross-reference; it does not query the WO module's tables directly. All required signals come through the Kafka events `WorkOrderFinalized` and `WorkOrderCancelled`.

customer_order_processed_workorders — IDEMPOTENCY

Sample row added by this step:

order_id	work_order_id	kind	processed_at
`ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1`	`wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2`	`install`	2026-05-18 15:35:00.789

8. Kafka events emitted

sophix.orders.install-dispatched

```
{
  "eventType":   "OrderInstallDispatched",
  "aggregateId": "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
  "timestamp":   "2026-05-18T12:00:00.234Z",
  "payload": {
    "orderId":               "ord_01HZX9K8M7Q2N4P6R3T5W8Y0Z1",
    "customerId":            "cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6",
    "operatorCode":          "WIK",
    "subscriptionId":        "sub_01HX5M2K9P3Q7R4T6W8Y0Z2N4Q",
    "workOrderId":           "wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2",
    "homepassId":            "hp_01HX5M2K9P3Q7R4T6W8Y0Z2N4P",
    "packageRef":            "pkg_INET_VOIP_RES",
    "equipmentRequirements": [
      { "type": "GPON_ONT", "mandatory": true },
      { "type": "ATA",      "mandatory": true }
    ],
    "scheduledFor":          "2026-05-18T15:00:00Z",
    "installTimeoutAt":      "2026-06-17T12:00:00Z",
    "dispatchedAt":          "2026-05-18T12:00:00.234Z"
  }
}
```

sophix.orders.install-failed

```
{
  "eventType":   "OrderInstallFailed",
  "aggregateId": "ord_01HZ...C",
  "timestamp":   "2026-06-17T12:00:00.234Z",
  "payload": {
    "orderId":              "ord_01HZ...C",
    "customerId":           "cust_01HX...C",
    "operatorCode":         "WIK",
    "subscriptionId":       "sub_01HX...C",
    "workOrderId":          "wo_01HZ...C",
    "failureReason":        "INSTALL_TIMEOUT",
    "failureDetail":        "Contractor did not finalize within PT30D",
    "paymentReceivedAmount":15000.00,
    "currency":             "KES",
    "refundRequested":      true,
    "failedAt":             "2026-06-17T12:00:00Z"
  }
```

```
}
```

Consumers:

Consumer	Purpose
DD_NOT-01	On `OrderInstallDispatched`: customer SMS with install date/time-window. On `OrderInstallFailed`: customer SMS with refund timeline.
BIL-02-ADJ-01	Refund queue on `OrderRefundRequested`.
CVM	Funnel install stage tracking + failure rate by region.
Finance	Refund processing.
NOC	Alerts on PT30D timer fires (operator delivery SLA breach).

9. Step-pinned rules

ID	Rule
R-FUL-02-INS-1	`order-create-install-wo` resolves equipment requirements by walking the Package's `serviceRefs[]` (SIP-01) → for each Service, GET PLM-CFG-01 → collect `equipmentRequirementRef` where `isAddressable=true`. The aggregated list goes into the WO payload.
R-FUL-02-INS-2	`WorkOrderFinalizedConsumer` writes `equipment_bindings_captured` onto the order row AND passes `equipmentBindings` as a Camunda process variable on correlation. STEP-ACTIVATION reads the variable to forward to SUB-WF-ACTIVATE-01.
R-FUL-02-INS-3	PT30D boundary timer (operator-tunable). On fire → `order-mark-install-failed` with `failureReason = INSTALL_TIMEOUT`. Timer is interrupting; late `WorkOrderFinalized` arrivals after timer fire → DLQ.
R-FUL-02-INS-4	`WorkOrderCancelled` correlation handling is operator-BPMN-defined. v1.0 WIK auto-fails the order (`failureReason = WORK_ORDER_CANCELLED`). Future operators may interpose a manual-investigation user task instead.
R-FUL-02-INS-5	INSTALL_FAILED with `payment_received_amount > 0` triggers refund cascade — `order-mark-install-failed` emits both `OrderInstallFailed` and `OrderRefundRequested`. Same refund attribution pattern as STEP-ACTIVATION (separate events for separate failure causes).
R-FUL-02-INS-6	At-least-once delivery: `customer_order_processed_workorders(order_id, work_order_id)` UNIQUE constraint blocks duplicate handling.
R-FUL-02-INS-7	Equipment-binding integrity: `WorkOrderFinalized` event MUST include a binding for every mandatory equipment requirement carried in the WO's `metadata.equipmentRequirements`. Missing-binding event → consumer fails the message correlation (returns failure to Kafka, retries), eventually ops sees the issue. Server-side enforcement of binding completeness is owned by the WO module's `wo-capture-equipment-bindings-at-finalize` worker (DD_WO-01-FLOW-INSTALLATION §7.2): per-job-type required-bindings rules block second-confirm if mandatory bindings are missing.
R-FUL-02-INS-8	Subscription MUST exist before STEP-INSTALL runs. In the KE WIK BPMN this is guaranteed by STEP-SUBSCRIPTION running early (post-validate, pre-payment). The worker validates `subscriptionId IS NOT NULL` on the input; missing → `/failure` with `MISSING_SUBSCRIPTION` (BPMN composition bug — should not happen in WIK).